Course: SWE 4404 – Software Project Lab II

WANDERVILLE

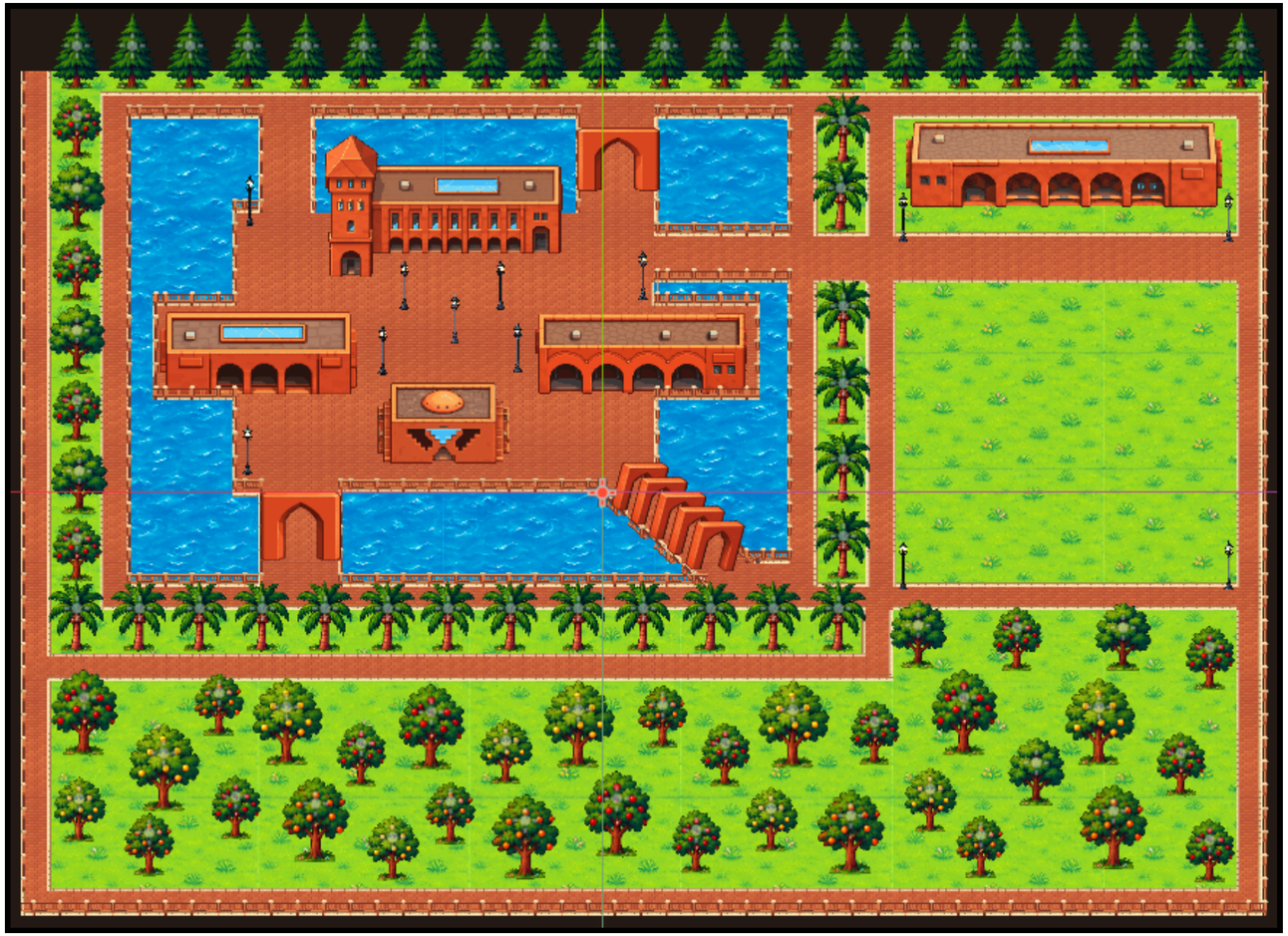
FINAL PRESENTATION



TEAM MEMBERS

Safia Aman Anika	230042120
Mushfeka Ferdousi	230042126
Khadiza Sultana	230042135

PROJECT OVERVIEW



1. Wanderville is a 2D multiplayer farming game — blending farming, fishing, pets, and social play.

2. Real-time multiplayer game with proximity text chat with friends

AUTHENTICATION

If an account already exists,the player skips login and directly enters the world

SIGN UP



LOG IN



MULTIPLAYER SYSTEM

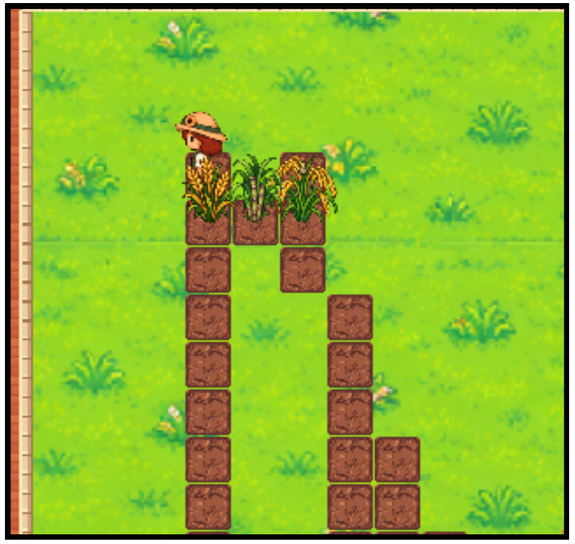


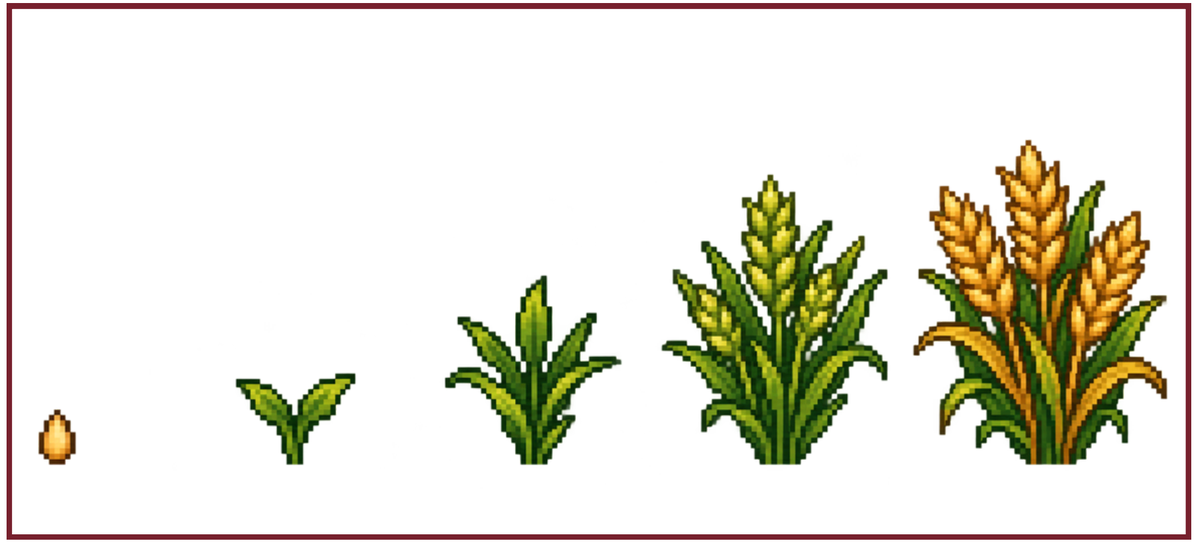
1. Friends can communicate via text
2. Friend request via username search
3. Friends can send and receive gifts

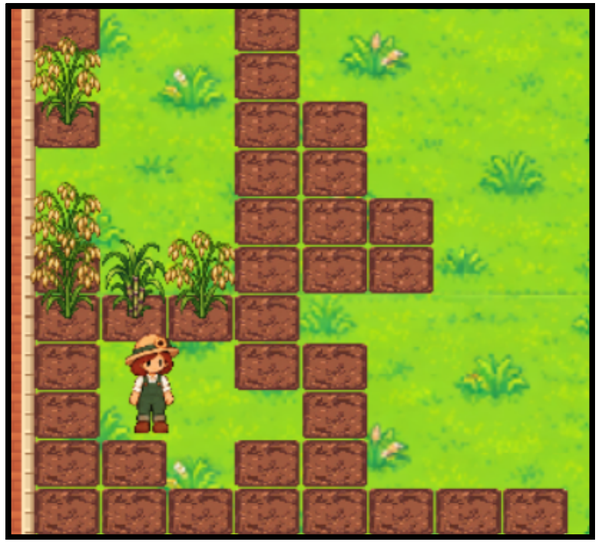




FARMING SYSTEM







1. Grid-based farming where each tile = **1 crop slot**
2. Harvest: earns **Aura** and adds to **inventory**

FISHING SYSTEM





Players have to catch the fish within 2 minutes to win

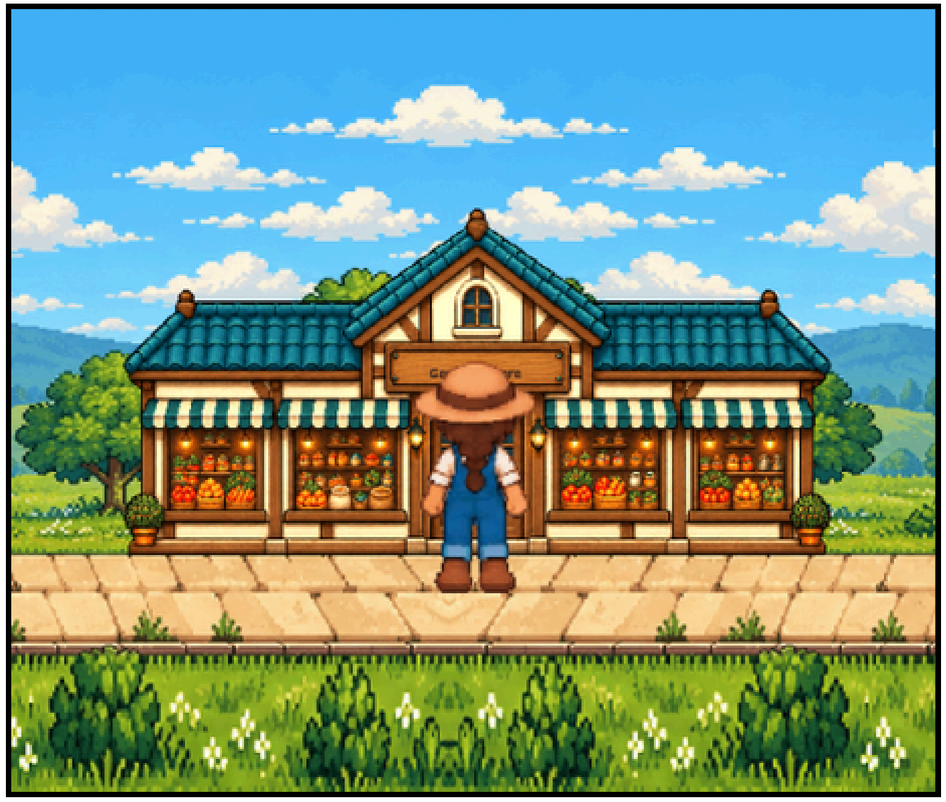
HOUSING SYSTEM





Cut trees → **Collect wood** → **Build house** → **Decorate with furniture**

MARKETPLACE





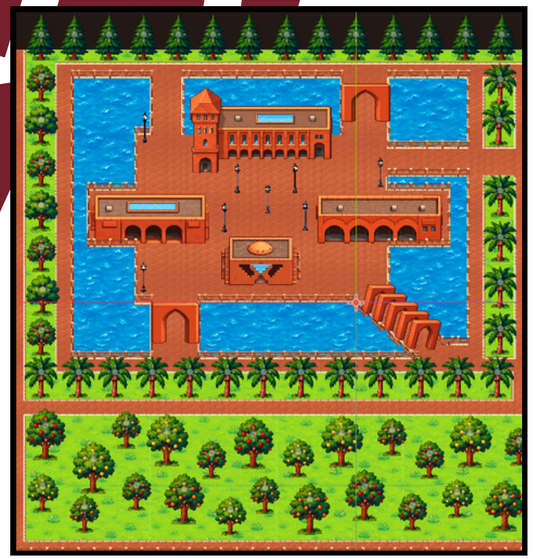
Players can buy crops from marketplace and sell crops from their inventory

PET COMPANION



Players can choose a furry pet as a cute companion in the game

WORLD SYSTEM



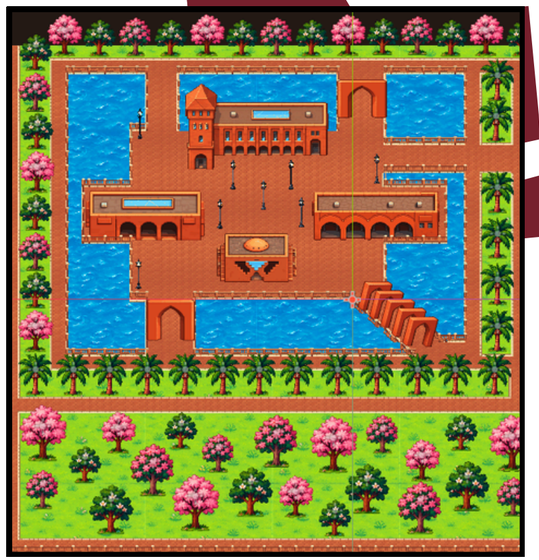
Summer



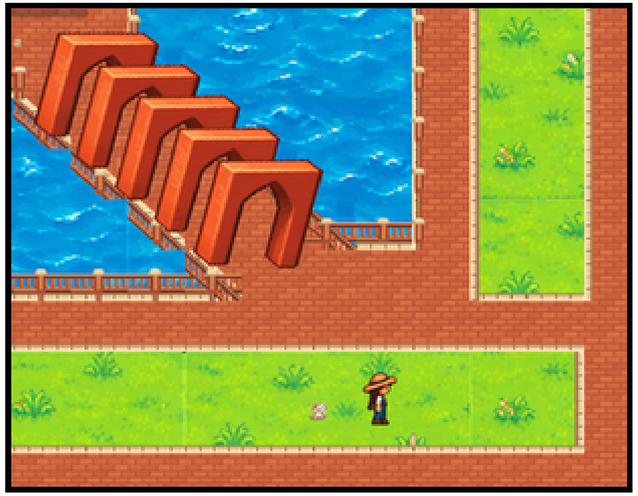
Autumn

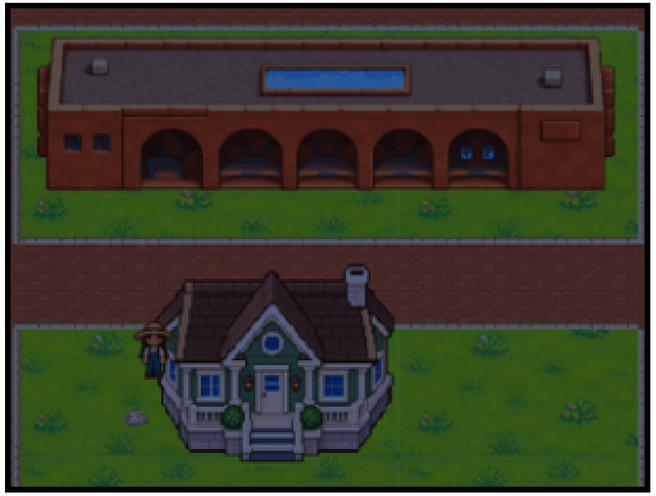


Winter



Spring

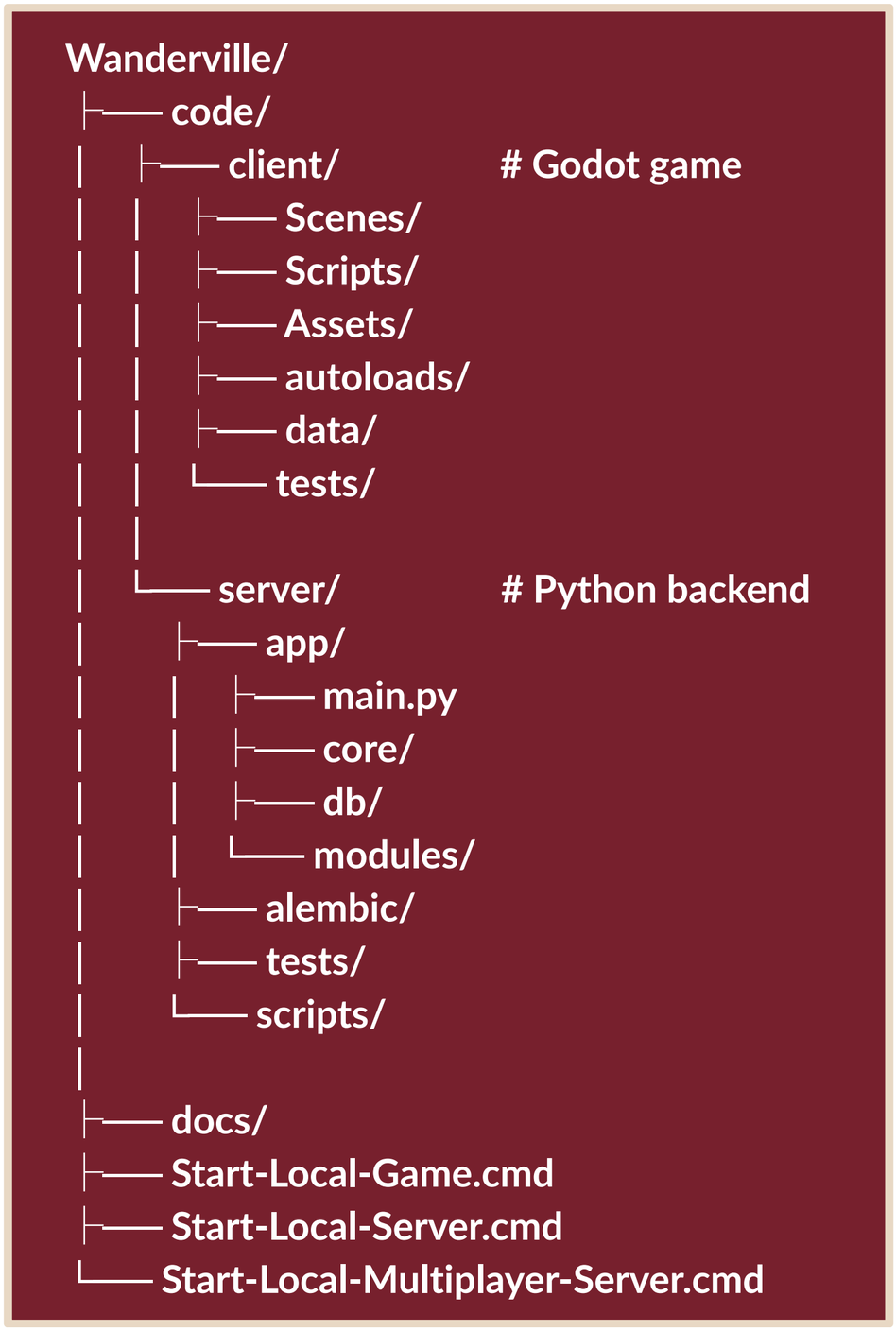




Day: 25 mins | **Night:** 5 mins

Seasons: Every 24 hours

Rain during summer and snowfall during winter



PROJECT STRUCTURE

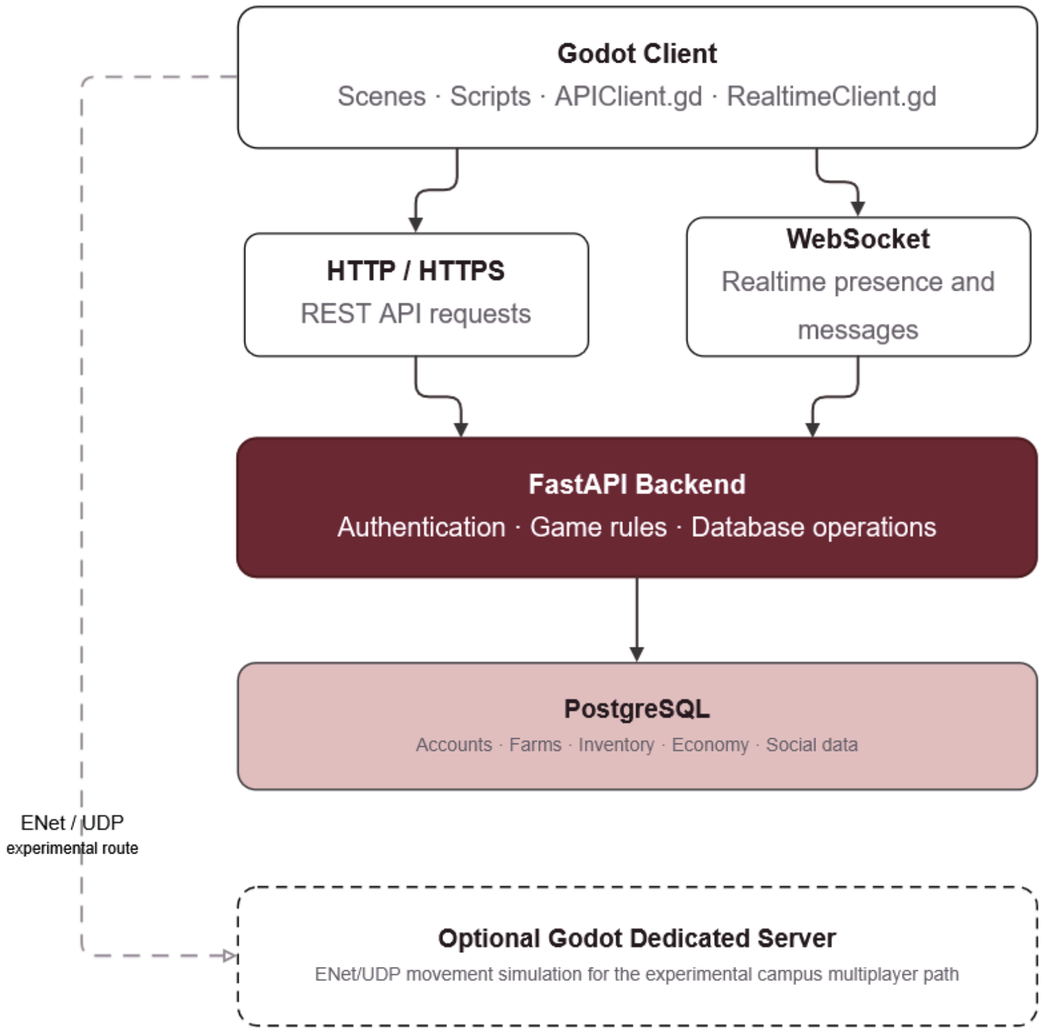
Client	Godot scenes, gameplay scripts, UI, assets and client-side networking.
Server	FastAPI application, gameplay business logic, database access and API endpoints.
Modules	Separate gameplay domains such as farming, inventory, economy, authentication and social interactions.
Tests	Verify behaviors.
Docs	Support development, testing, deployment and onboarding.

LAYERED BACKEND ARCHITECTURE



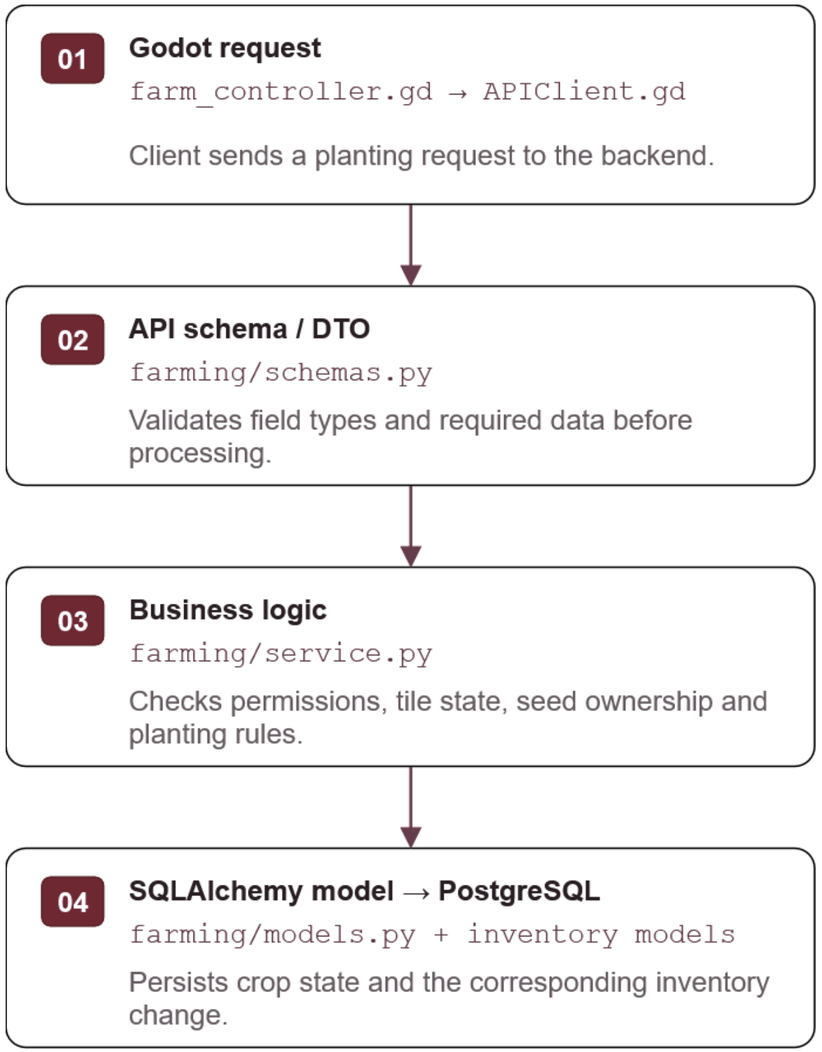
- main.py assembles the application and registers routers.
- core/ contains shared configuration, security and application-level utilities.
- db/ contains database setup, sessions and model registration.
- modules/ groups related business features.
- alembic/ manages schema migrations.
- tests/ verifies individual features and cross-component behavior.

HOW THE CLIENT, SERVERS AND DATABASE COMMUNICATE



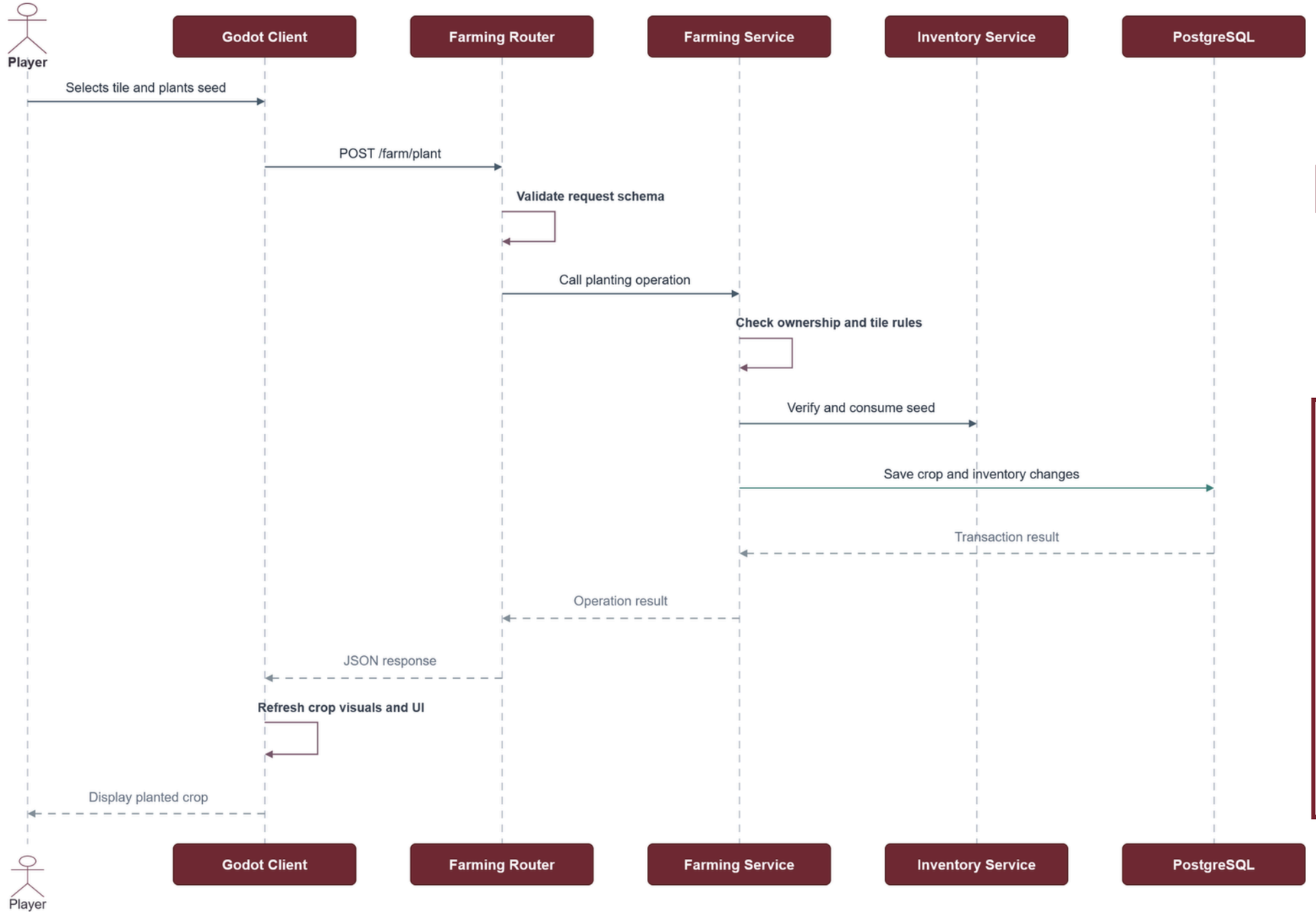
- **Godot → FastAPI:** JSON requests over HTTP; authenticated requests include the appropriate access token.
- **FastAPI → PostgreSQL:** SQLAlchemy and the configured database driver execute database operations.
- **FastAPI ↔ Godot:** JSON responses carry results and updated state.
- **Godot ↔ FastAPI:** WebSocket messages support realtime functionality.
- **Godot ↔ dedicated server:** ENet/UDP for the optional movement path.
- **Dedicated server ↔ FastAPI:** Internal HTTP communication for admission and position projection, where configured.

DATA EXCHANGE AND PERSISTENCE



- Godot Client: Sends requests to the FastAPI backend.
- API Schema / DTO: Validates and structures incoming data using Pydantic.
- Business Logic: Processes requests, enforces rules, and handles operations.
- Database Persistence: SQLAlchemy manages data storage and retrieval in PostgreSQL.

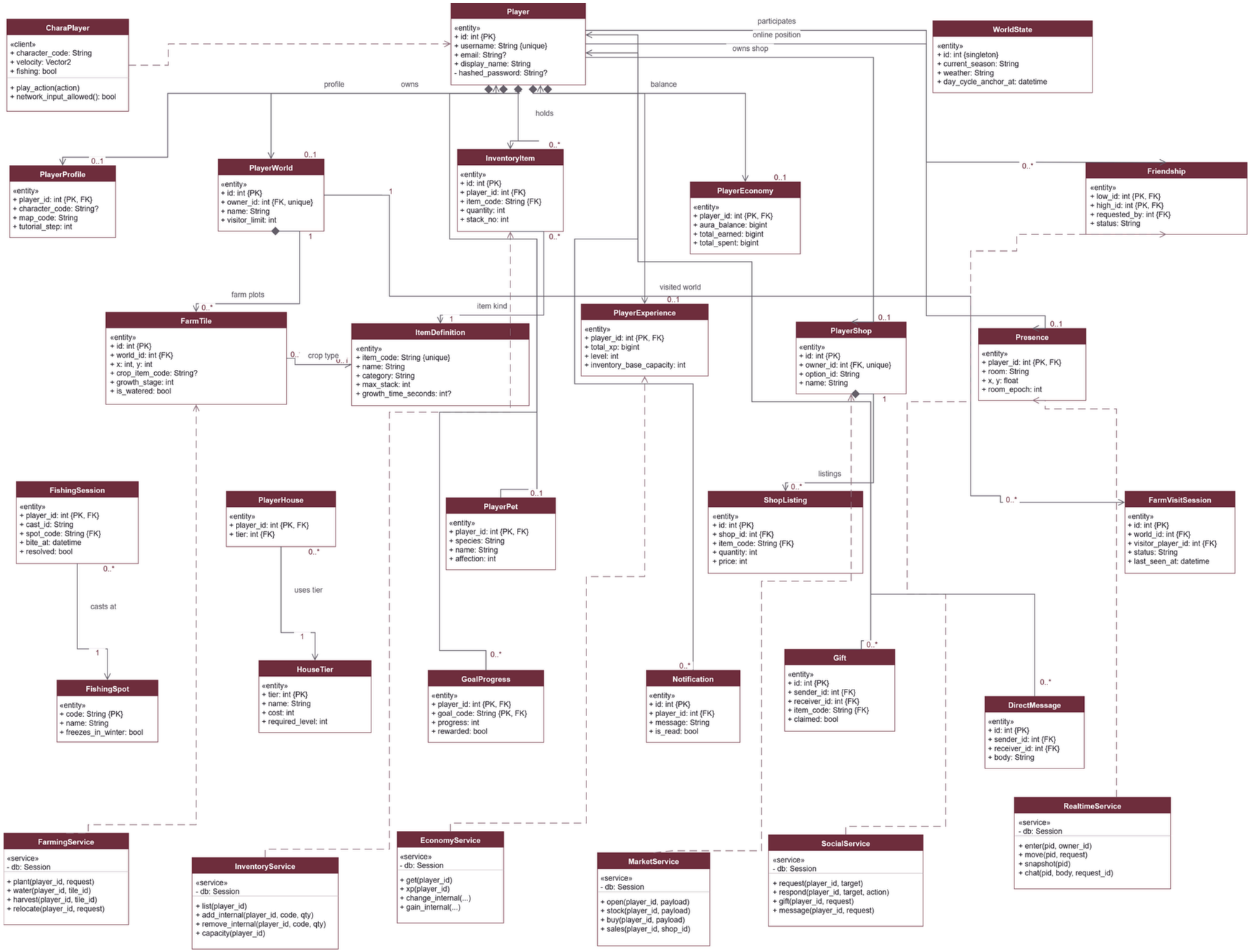
END-TO-END FEATURE REQUEST



Crop Planting Sequence Diagram

Illustrates the flow of a crop-planting request from the Godot client through FastAPI services to PostgreSQL, including validation, data updates, and UI synchronization.

CLASS DIAGRAM

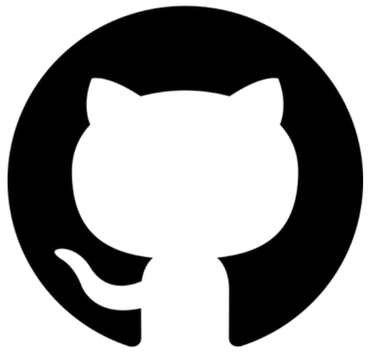


MAXIMUM AND AVERAGE LINES OF CODE

Functions / methods
Count: 641 **Maximum LOC:** 185 (db\gameplay_seed.py:seed_gameplay) **Average LOC:** 11.89

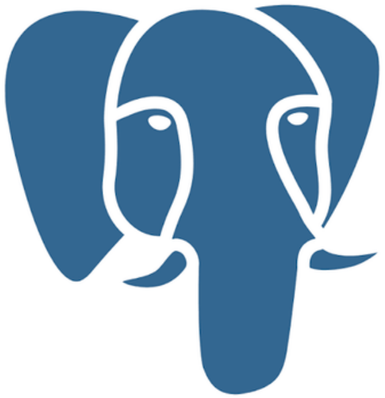
Classes
Count: 191 **Maximum LOC:** 476 (modules\realtime\service.py:RealtimeService) **Average LOC:** 33.90

TOOLS & TECHNOLOGIES





SQLAlchemy









WebSocket

FUTURE DIRECTIONS

1	Proximity text chat and voice
2	Visiting a friend’s farm
3	Adding animals that give different produce (milk, wool etc)
4	Shops that sell various types of food
5	Improving character customization to wear different dresses
6	Adding factories (rice-making, sugar-making etc)

THANK YOU

